

Chapter: Matrix & Grid Pathfinding

Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 19, 2026

Problem Approach: Minimum Path Sum

1. Queue-based Traversal (BFS)

Instead of strict recursive DP, we utilize a queue to traverse the matrix starting from $\{0, 0\}$. This systematically explores possible paths cell by cell, allowing us to compute minimum distances progressively.

2. Allowed Directions

We are restricted to moving only **Right** or **Down**. This is managed by a direction vector array: $\text{dir} = \{\{1, 0\}, \{0, 1\}\}$ to calculate neighboring coordinates cleanly.

3. Cost Tracker (minu matrix)

A 2D vector `minu` is initialized with `INT_MAX` to store the minimum cost to reach any specific cell (r, c) . The base cell `minu[0][0]` is seeded with `grid[0][0]`.

4. State Relaxation

At each step, we evaluate adjacent valid cells. If moving to a neighbor yields a smaller accumulated sum, we update `minu[nr][nc] = min(minu[nr][nc], minu[r][c] + grid[nr][nc])`.

Visualizing the Traversal & Cost Update Matrix

S	→	
↓		
		E

Start (S) to End (E) relaxing weights dynamically.

Matrix bounds check: `if(nr<0 || nc<0 || nr>=n || nc>=m) continue;`

Summary of Complexity: The seen matrix prevents nodes from being redundantly pushed into the queue multiple times in the initial exploration, while the minimum values dynamically update. The final answer rests in `minu[n-1][m-1]`.

C++ Source Code

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {

        int n = grid.size();
        int m = grid[0].size();

        queue<pair<int, int>> q;
        vector<vector<int>> seen(n, vector<int>(m, 0));
        vector<pair<int, int>> dir = {{1, 0}, {0, 1}};

        q.push({0, 0});
        seen[0][0] = 1;
        vector<vector<int>> minu(n, vector<int>(m, INT_MAX));
        minu[0][0] = grid[0][0];

        while(!q.empty()){
            auto curr = q.front();
            q.pop();

            int r = curr.first;
            int c = curr.second;

            for(auto &it : dir){
                int nr = r + it.first;
                int nc = c + it.second;

                if(nr < 0 || nc < 0 || nr >= n || nc >= m) continue;

                minu[nr][nc] = min(minu[nr][nc], minu[r][c] + grid[nr]
[nc]);

                if(!seen[nr][nc]){
                    seen[nr][nc] = 1;
                    q.push({nr, nc});
                }
            }
        }

        return minu[n-1][m-1];
    }
};
```